

26년 경북대 AX Intensive 교육

1주차 – pandas 기본



소 속 : 경북대학교/인공지능학과

이 름 : 김현철 교수

E-Mail : hyunchul_kim@knu.ac.kr

Timetable

시간표		학습 내용
1차시	09:30 – 09:50	OT (30분)
2차시	10:00 – 10:50	Pandas 기본 (이론/실습)
3차시	11:00 – 11:50	Pandas 기본 (실습)
Lunch Break (11:50 – 13:30)		
5차시	13:30 - 14:20	개인별 AX 과제 정의(발표) 및 랩실 멘토링
6차시	14:30 – 15:20	
7차시	15:30 – 16:20	
8차시	16:20 – 17:30	

1. Packages

- <https://numpy.org/>



- <https://pandas.pydata.org/>

표 형식 데이터의 개념

1. 표 형식 데이터의 개념

- 표 형식 데이터란?
 - 행(row)과 열(column)로 구성된 2차원 구조의 데이터
 - 각 행은 하나의 관측치(observation), 각 열은 하나의 변수(variable)를 나타냄
 - 일상생활 예시: 학생 성적표, 회사 직원 명단, 온라인 쇼핑몰 상품 목록
- 표 형식 데이터의 장점
 - 직관적이고 이해하기 쉬운 구조
 - 다양한 데이터 타입을 한 번에 저장 가능
 - 데이터 분석 도구들이 표준적으로 지원

1. 표 형식의 데이터 다루기

- 표의 주요 구성요소
 - 행 (row)
 - 열 (column)
 - 셀 (cell)
 - 헤더(레이블) (header, label)
 - 인덱스 (index)

1.

데이터는 행 단위로 채워진다

	total_bill	tip	sex	smoker	day	time	size
1	27.20	4.00	Male	No	Thur	Lunch	4
2	20.27	2.83	Female	No	Thur	Lunch	2
3	16.00	2.00	Male	Yes	Thur	Lunch	2
4	13.16	2.75	Female	No	Thur	Lunch	2
5	16.82	4.00	Male	Yes	Sun	Dinner	2
6	23.17	6.50	Male	Yes	Sun	Dinner	4



7	18.15	3.50	Female	Yes	Sun	Dinner	3
---	-------	------	--------	-----	-----	--------	---

1.

데이터 타입

Pandas	Python	사용
Object	str	텍스트 또는 혼합 숫자 숫자가 아닌 값
Int64	Int	정수형 숫자
Float64	Float	부동소수점(실수형) 숫자
Bool	Bool	True/False
Datetime64	NA	날짜 및 시간
Timedelta64[na]	NA	두 날짜/시간 사이의 차이
category	NA	범주형 데이터

pandas 기초

2. Pandas 소개

- 구조화된 데이터나 표 형식 데이터를 빠르고 쉽게 다루도록 설계
- 고수준의 자료구조와 데이터 분석 도구 포함
- NumPy의 배열 기반 계산 스타일 차용
 - 반복문 없이 데이터 처리가 가능한 점
- 이름의 유래 : **panel data + Python data analysiss**

2. Pandas 소개

- pandas를 사용하는 이유
 - 엑셀보다 빠른 대용량 데이터 처리
 - 반복 작업의 자동화 가능
 - 다양한 데이터 형식 지원 (CSV, Excel, JSON, SQL 등)
 - 강력한 데이터 분석 및 시각화 기능
- pandas의 핵심 개념
 - 벡터화 연산: 반복문 없이 전체 데이터에 연산 적용
 - 인덱스 기반 데이터 접근
 - 결측치 처리의 용이성

2. pandas 설치

- **Installation**

```
pip install pandas
```

- colab 에는 설치되어 있음

2. 모듈 import

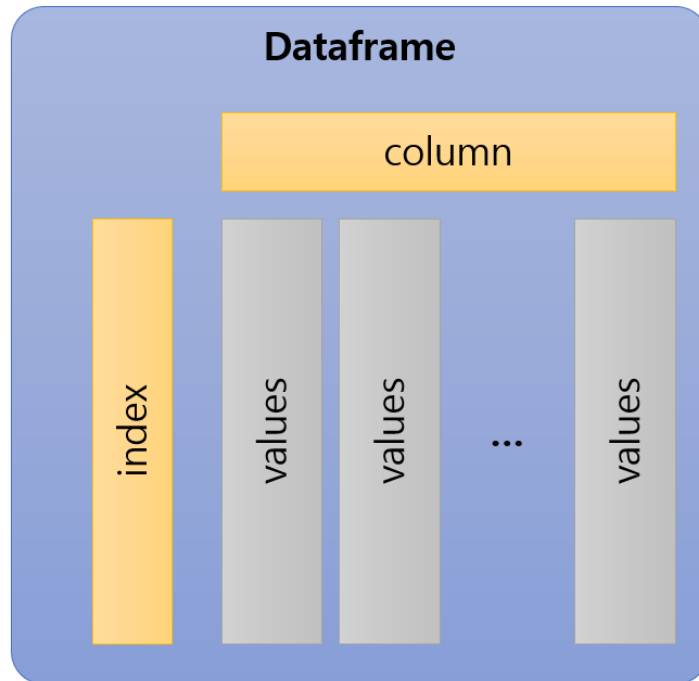
```
import numpy as np  
import pandas as pd  
pd.__version__
```

→ '3.0.4'

2.

핵심 객체

- Series
- Dataframe



Dataframe 객체

4. DataFrame

- DataFrame은 표 형식의 데이터 구조
 - 행과 열로 이루어진 2차원 배열
 - 엑셀 스프레드시트와 유사한 형태로 데이터를 조작할 수 있음
 - 다양한 유형의 데이터를 저장하고 처리하는 데 사용
 - 각 열은 데이터의 특정 유형(숫자, 문자열, 날짜 등)을 나타내는 Series 객체로 구성
- DataFrame의 구조
 - 여러 개의 Series가 열로 결합된 형태
 - 각 열은 서로 다른 데이터 타입 가능
 - 행과 열 모두에 인덱스 존재
- DataFrame vs 엑셀 시트
 - 공통점: 행과 열로 구성된 표 형태
 - 차이점: 프로그래밍을 통한 자동화, 대용량 처리 가능

4. Dataframe

- 표 (스프레드시트) 형식의 자료구조

columns

	total_bill	tip	sex	smoker	day	time	size
1	27.20	4.00	Male	No	Thur	Lunch	4
2	20.27	2.83	Female	No	Thur	Lunch	2
3	16.00	2.00	Male	Yes	Thur	Lunch	2
4	13.16	2.75	Female	No	Thur	Lunch	2
5	16.82	4.00	Male	Yes	Sun	Dinner	2
6	23.17	6.50	Male	Yes	Sun	Dinner	4

index values

4. DataFrame 생성하기

- From dict

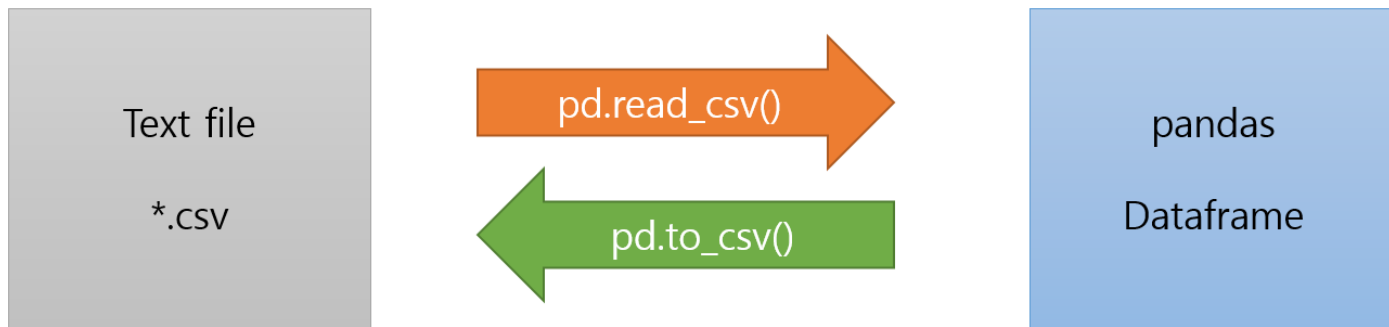
```
d = {"col1": [1, 2, 3, 4], "col2": [5, 6, 7, 8]}  
df = pd.DataFrame(d)
```

	col1	col2
0	1	5
1	2	6
2	3	7
3	4	8

데이터 로딩과 저장

5. pandas 파일 입/출력

- 파일을 읽어서 dataframe을 생성해주고, 파일에 쓰기 위한 함수가 파일 형식별로 존재함
 - 예) CSV



5. File로 부터 data 읽고 쓰기

- csv file

```
df = pd.read_csv('filename.csv')  
df.to_csv('output.csv')
```

- excel file

```
df = pd.read_excel('filename.xlsx')    # pip install xlrd  
df.to_excel('output.xlsx')            # pip install openpyxl
```

Dataset 탐색을 위한 기본 함수

6. Dataset 준비

```
import seaborn as sns  
df = sns.load_dataset('tips')
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

6. head(), tail(), shape를 활용한 데이터 구조 확인

- `df.head()`
 - 처음 5개 행 출력
- `df.tail()`
 - 마지막 5개 행 출력
- `df.shape`
 - DataFrame의 행과 열 개수를 (행 수, 열 수) 형태로 변환

6. Descriptive statistics (기술 통계량) 확인

- describe()
 - 수치형 데이터의 분포 파악
 - 숫자형 데이터에 대해서만 적용됨

```
df.describe()
```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

6. summary of DataFrame

- info()
 - 메모리 사용량, 결측치 확인

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 244 entries, 0 to 243
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	total_bill	244 non-null	float64
1	tip	244 non-null	float64
2	sex	244 non-null	category
3	smoker	244 non-null	category
4	day	244 non-null	category
5	time	244 non-null	category
6	size	244 non-null	int64
7	md	0 non-null	float64

```
dtypes: category(4), float64(3), int64(1)
```

```
memory usage: 9.3 KB
```

6. 특정 컬럼의 기본 통계값 확인

- `df['total_bill'].mean()`
 - 평균값 확인
- `df['total_bill'].max()`
 - 최댓값 확인
- `df['total_bill'].idxmax()`
 - 최댓값을 가진 행의 인덱스 확인

→ `max()` 는 최댓값을 반환하고, `idxmax()`는 최댓값을 가진 행의 인덱스를 반환합니다.

6. 범주형 자료

- unique()
 - 고유값 종류 파악
 - 범주형 자료 종류 확인

```
df['day'].unique()
```

```
['Sun', 'Sat', 'Thur', 'Fri']
```

```
Categories (4, object): ['Thur', 'Fri', 'Sat', 'Sun']
```

- value_counts()
 - 범주형 데이터의 빈도 확인

```
df['smoker'].value_counts()
```

```
smoker
```

```
No          151
```

```
Yes          93
```

```
Name: count, dtype: int64
```

6. 컬럼 선택

- `dataframe['컬럼 이름']`

```
df['day'], df.smoker  
df[['day', 'smoker']]
```

	0	1	2	3	4	5	6
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

6. 컬럼 추가 (기존 컬럼으로 부터)

- `dataframe['새 컬럼 이름'] = data`

```
df['ind_bill'] = df['total_bill'] / df['size']
```

또는

```
df.assign(ind_bill=df['total_bill'] / df['size'])
```

6. Column 삭제

```
del dataframe['컬럼 이름']  
del df['ind_bill'] # 추가한 컬럼 삭제  
  
df.drop('size', axis=1)
```

6. Row 추가

- Append (단일 값으로 마지막에 추가)

```
df.loc[244] = np.nan
```

- List로 추가

```
df.loc[245] = [18.78, 3.00, 'Female', 'No', 'Thur',  
              'Dinner', 2.0]
```


6. Row 삭제

- Row 삭제

Drop ()

```
df.drop(0)
```

```
df.drop([1,2,4])
```

```
df.drop(np.arange(5))
```

```
df.drop(0, inplace=True) # 원본 데이터에 적용!
```

데이터 인덱싱과 슬라이싱

7. pandas에서의 슬라이싱 특징

- pandas 슬라이싱의 특징
 - 행(row) 슬라이싱과 열(column) 슬라이싱 모두 가능
 - 정수 인덱스와 라벨 인덱스 모두 지원
 - 조건부 슬라이싱으로 필터링과 결합 가능

```
df[1:4]      # 1번째부터 3번째 행까지 선택
df[:3]       # 처음부터 2번째 행까지 선택
df[2:]       # 2번째 행부터 끝까지 선택
df[:, :2]    # 모든 행을 2개씩 건너뛰며 선택
```

7. 인덱싱 vs 슬라이싱

- 인덱싱 vs 슬라이싱
 - 인덱싱: 특정 위치의 단일 요소 선택
 - 슬라이싱: 연속된 범위의 여러 요소 선택

인덱싱 (단일 선택)

`df.loc[0]`

0번째 행만 선택 → Series 반환

`df['tip']`

tip 열만 선택 → Series 반환

슬라이싱 (범위 선택)

`df.loc[0:2]`

0~2번째 행 선택 → DataFrame

반환

`df[['tip', 'size']]`

여러 열 선택 → DataFrame 반환

7. loc와 iloc의 기본 개념

- loc: 라벨(Label) 기반 인덱싱
 - 행과 열의 이름(라벨)을 사용하여 데이터 선택
 - 인덱스와 컬럼명으로 데이터에 접근
- iloc: 정수 위치(Integer Location) 기반 인덱싱
 - 행과 열의 숫자 위치를 사용하여 데이터 선택
 - 0부터 시작하는 정수 인덱스로 데이터에 접근
- 기억하기 쉬운 방법
 - loc = Location by Label (라벨로 위치 찾기)
 - iloc = Integer Location (정수로 위치 찾기)

7. 슬라이싱에서의 중요한 차이점

- loc 슬라이싱: 끝 인덱스 포함 (inclusive)
 - `df.loc[0:3]` → 인덱스 0, 1, 2, 3 모두 포함
- iloc 슬라이싱: 끝 인덱스 제외 (exclusive)
 - `df.iloc[0:3]` → 인덱스 0, 1, 2만 포함 (3은 제외)

<code>df.loc[1:3, 'tip']</code>	# 인덱스 1, 2, 3의 tip 값
<code>df.iloc[1:3, 1]</code>	# 1, 2번째 행의 1번째 열 값

7. 사용 예시

- loc 사용 예시 (라벨 기반)
 - 단일 행 선택: `df.loc[0]`
 - 여러 행 선택: `df.loc[0:2]`
 - 특정 컬럼 선택: `df.loc[:, 'tip']`
 - 행과 열 동시 선택: `df.loc[0:2, 'tip':'size']`
 - 조건부 선택: `df.loc[df['tip'] > 3, 'sex']`
- iloc 사용 예시 (위치 기반)
 - 단일 행 선택: `df.iloc[0]`
 - 여러 행 선택: `df.iloc[0:3]`
 - 특정 컬럼 선택: `df.iloc[:, 1]`
 - 행과 열 동시 선택: `df.iloc[0:3, 1:4]`
 - 간격을 두고 선택: `df.iloc[::2, ::2]`

데이터 필터링

8. 조건에 맞는 행 선택: 필터링

- 조건을 활용한 필터링

- 조건 연산자 (==, >, <, >=, <=, !=)를 사용
- 단일 조건: `df[df['column'] > value]`

```
df[df['tip'] >= 3] # tip이 3달러 이상인 행 선택  
df[df['tip'] <= 3] # tip이 3달러 이하인 행 선택
```

- 여러 조건의 결합

- 논리 연산자 (&, |)를 사용하여 여러 조건을 결합
- 복합 조건: `df[(조건1) & (조건2)]`
- 주의: and/or 대신 &| 사용

```
df[(df['tip'] > 3) & (df['sex'] == 'Male')]  
# tip이 3달러 초과이고 성별이 남성인 행만 선택
```

8. 다양한 함수 활용

- `isin()`
 - 개념: 특정 값들이 포함된 행을 선택하는 함수
 - 사용법: `df[df['컬럼명'].isin([값1, 값2, ...])]`

```
df[df['day'].isin(['Sat', 'Sun'])]  
# tip이 Sat 또는 Sun 행 선택
```

- `between()`
 - 개념: 특정 범위 내의 값을 가진 행을 선택하는 함수
 - 사용법: `df[df['컬럼명'].between(최솟값, 최댓값)]`

```
df[df['tip'].between(2, 5)]  
# tip이 2~5달러 사이인 행만 선택
```

- `str.contains()`
 - 개념: 문자열 컬럼에서 특정 문자나 패턴을 포함한 행을 선택
 - 사용법: `df[df['컬럼명'].str.contains('검색할문자')]`

```
df[df['day'].str.contains('S', na = False)]  
# day 컬럼에 'S'가 포함된 요일만 선택
```

8. `iloc`, `loc`를 활용한 행/열 선택

- `iloc` (정수 위치 기반 선택)
 - 0부터 시작하는 정수 인덱스 사용
 - 행과 열의 위치로 데이터 선택

```
df.iloc[0:3] # 0~2번째 행 선택
```

- `loc` (라벨 기반 선택)
 - 인덱스와 컬럼명으로 데이터 선택

```
df.loc[0:3, 'tip'] # 인덱스 0~3번의 tip 컬럼만 선택
```

- 조건과 결합
 - 필터링과 동시에 특정 컬럼만 선택

```
df.loc[df['tip'] > 3, ['sex', 'tip']]  
# tip이 3초과인 행의 sex, tip 컬럼만 선택
```

데이터 정렬

9. 데이터 정렬

- 데이터 정렬이란?
 - 데이터를 특정 기준에 따라 순서대로 배열하는 과정
 - 데이터 분석의 기본이 되는 중요한 전처리 작업
- 정렬의 활용 사례
 - 상위/하위 데이터 확인 (매출 순위, 성적 순위 등)
 - 데이터 패턴 파악 및 이상치 발견
 - 시계열 데이터의 시간순 정렬
 - 그룹별 분석을 위한 사전 정렬

9. 단일 컬럼 정렬

- 단일 컬럼 정렬이란?
 - 하나의 컬럼을 기준으로 전체 데이터를 정렬하는 방법
 - 가장 기본적이고 자주 사용되는 정렬 방식
 - 숫자형과 문자형 데이터 모두 정렬 가능
- `sort_values()` 기본 문법
 - `df.sort_values(by='컬럼명')`
 - `by`: 정렬 기준이 될 컬럼명 지정
 - `ascending`: 정렬 방향 (`True`=오름차순, `False`=내림차순)

```
df.sort_values(by='tip') # 기본 오름차순 정렬
```

```
df.sort_values(by='tip', ascending=False) # 내림차순 정렬
```

```
df.sort_values(by='day') # 문자형 컬럼 정렬 (알파벳 순)
```

```
# 정렬 결과를 원본에 적용
```

```
df.sort_values(by='tip', inplace=True)
```

데이터 정제

10. 데이터 정제 및 준비

- 주어진 데이터를 구조화하고 분석에 적합하도록 준비하는 과정
 - 데이터 불러오기
 - 데이터 정제하기
 - 데이터 변형하기
 - 데이터 재정렬
- 목적: 데이터를 원하는 형태로 가공하는 것!

10. 데이터 준비/정제 관련 주제

- 결측 데이터 처리 (Handling missing data)
- 데이터 변형 (Data transformation)

10. 결측치 처리

- 수집된 데이터에 누락된 값이 존재할 경우
 - 해당 데이터를 삭제
 - 다른 값으로 채움
- `isna()`
- `fillna()`
- `dropna()`

10. 결측치 확인

- 결측치는 numpy 의 nan 상수로 처리됨

```
df['md'] = np.nan      # 임의로 결측치 생성
df.isna()              # 각 셀 값이 NaN인지 T/F로 반환
df.isna().sum()        # 각 셀에서 결측치가 몇 개 인지 확인
```

10. 결측 데이터 제외하기 : dropna()

```
data = pd.DataFrame([[1., 6.5, 3.],  
                    [1., np.nan, np.nan],  
                    [np.nan, np.nan, np.nan],  
                    [np.nan, 6.5, 3.]])
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

- dropna()
 - 하나라도 NaN이 포함된 행 제외
 - thresh : 개수 이상인 행만 남김
 - how='all' 로 주면, 모든 컬럼의 값이 nan인 행을 삭제
- Isna()
 - 하나라도 NaN이 포함된 행 제외

10. 결측치 채우기

```
df['column_name'].fillna(50) # 특정값으로 채우기  
df.ffill # 이전 행의 값으로 채움  
df.fillna({'tip':0., 'sex':'Male'}) # 열마다 다른 값 채우기  
df.fillna(data.mean()) # 각 열 평균값으로 채우기
```

Thank you

Q & A

